From Flow Matching to MeanFlow

The one-equation step that turns a many-step velocity field into a one-step *average*-velocity field

Desmond Cheong
Master's thesis — Taiwan-domain StormCast distillation

June 24, 2026

Goal: introduce the MeanFlow training objective the way I present it in the thesis — as a single, well-motivated generalisation of the straight-line flow-matching objective the FlowCast baseline already uses, not a new method dropped from the sky. Notation follows the methods chapter (Chapter 3, § *The MeanFlow Residual*) and the implementation in `stormcast/utils/meanflow_loss.py`. As there, all three heads model the *residual* $R_t = X_t - M_t$ left by the frozen regression mean M_t, in a space standardized by σ_{data}, conditioned on $c_t = \text{concat}(X_{t-1}, M_t, I)$; the flow time runs from *noise* to *data*.

1 Where we start: straight-line flow matching (FlowCast)

The FlowCast baseline is Independent Conditional Flow Matching on the standardized residual. With data endpoint $x_1 = R_t/\sigma_{\text{data}}$ and prior $x_0 \sim \mathcal{N}(0, I)$, the straight-line interpolant and its (constant) velocity are

$$x_t = (1-t)\,x_0 + t\,x_1 + \sigma_{\text{path}}\,\eta, \qquad v = x_1 - x_0, \tag{1}$$

and the network is regressed straight onto that velocity,

$$\mathcal{L}_{\text{FC}} = \mathbb{E}_{t,\,x_0,\,x_1}\,\big\|\, v_\theta(x_t, t; c_t) - (x_1 - x_0)\,\big\|^2_{2,\beta}\;(+\,\lambda_{\text{spec}}\mathcal{L}_{\text{spec}}), \tag{2}$$

with the precipitation-aware channel weights $\beta = (1, 1, 1, 2)$ and a shared `qpepre` log-PSD term that all heads carry (kept implicit from here on). The catch is the *sampler*: knowing only the instantaneous velocity, generation must integrate it,

$$x_{t+\Delta t} = x_t + \Delta t\; v_\theta(x_t, t; c_t), \qquad \Delta t = 1/K, \tag{3}$$

and because the learned *marginal* field is curved, a single Euler jump overshoots: accuracy needs $K \approx 10$ small steps per forecast hour. MeanFlow removes exactly this integration.

2 The one idea: predict the *average* velocity

Write the same straight-line path as the pure interpolant $z_s = (1-s)\,x_0 + s\,x_1$ (MeanFlow drops FlowCast's small σ_{path} thickening, $\sigma_{\text{path}} = 0$) and pick two flow times $r \le t$ along it. Define the **average velocity** over $[r, t]$,

$$u(z_r, r, t) \;\triangleq\; \frac{1}{t-r}\int_r^t v(z_\tau, \tau)\,\mathrm{d}\tau, \qquad \lim_{t\to r} u(z_r, r, t) = v(z_r, r), \tag{4}$$

a property of the underlying flow, just like v. Its defining feature is that transport over the interval becomes a single algebraic jump — no solver:

$$z_t \;=\; z_r + (t-r)\,u(z_r, r, t). \tag{5}$$

One evaluation at $(r,t) = (0,1)$ takes noise to data. The cost has moved from inference into training: (4) is a path integral we cannot evaluate, so u cannot be regressed directly.

3 Making it trainable: the MeanFlow identity

Eliminate the integral by differentiation rather than quadrature. Clear the denominator,

$$(t-r)\,u(z_r, r, t) \;=\; \int_r^t v(z_\tau, \tau)\,\mathrm{d}\tau, \tag{6}$$

and differentiate with respect to the *state* time r at fixed t, with z_r moving along the trajectory ($\mathrm{d}z_r/\mathrm{d}r = v$, $\mathrm{d}t/\mathrm{d}r = 0$). The product rule on the left and the fundamental theorem of calculus on the right give $-u + (t-r)\,\mathrm{d}u/\mathrm{d}r = -v$, i.e. the **MeanFlow identity**

$$\boxed{u(z_r, r, t) \;=\; v(z_r, r) \;+\; (t-r)\,\frac{\mathrm{d}u}{\mathrm{d}r}} \tag{7}$$

where $\mathrm{d}u/\mathrm{d}r$ is the *total* derivative along the trajectory — one Jacobian–vector product with tangent $(v, 1, 0)$ in (z, r, t):

$$\frac{\mathrm{d}u}{\mathrm{d}r} \;=\; v\,\partial_z u \;+\; \partial_r u \qquad (t \text{ fixed}). \tag{8}$$

The path integral is gone; the target needs only quantities at a single point of a single trajectory.

Convention note. This is Geng et al.'s identity transcribed to the thesis's noise-to-data convention. They place data at $s = 0$ and differentiate with respect to t, obtaining $u = v - (t-r)\,\mathrm{d}u/\mathrm{d}t$; here data sits at $s = 1$, the differentiation runs through the lower limit, and the sign flips — so do not line-diff (7) against the paper.

It contains flow matching exactly. Send the interval to zero. The prefactor $(t-r)$ vanishes and (7) collapses to the boundary condition

$$\lim_{r \to t} u(z_r, r, t) \;=\; u(z_t, t, t) \;=\; v, \tag{9}$$

the average velocity over a zero-width interval *is* the instantaneous velocity. On the diagonal $r = t$ the MeanFlow target is the flow-matching target: MeanFlow is a *strict generalisation* of FlowCast, not a different objective.

4 The training objective

Regress the network onto its own identity, holding the right-hand side constant with a stop-gradient $\mathrm{sg}[\cdot]$:

$$u_{\mathrm{tgt}} \;=\; \mathrm{sg}\Big[\, v + (t-r)\big(v\,\partial_z u_\theta + \partial_r u_\theta\big)\Big], \tag{10}$$

$$\mathcal{L}_{\mathrm{MF}} \;=\; \mathbb{E}\Big[\, \mathrm{sg}(w)\,\big\|\, u_\theta(z_r, r, t; c_t) - u_{\mathrm{tgt}}\,\big\|_{2,\beta}^2\Big], \qquad w = \big(\|\Delta\|_{2,\beta}^2 + \varepsilon_w\big)^{-p}. \tag{11}$$

The adaptive weight w (exponent $p = 1$, $\varepsilon_w = 10^{-3}$) down-weights high-error samples; $p = 0$ recovers plain weighted MSE. The derivative inside u_{tgt} is one forward-mode `torch.func.jvp` with tangent $(v, 1, 0)$, evaluated under `no_grad` on the bare module. Two flow times are drawn per sample and sorted to $r \le t$; a fraction $\rho_{\text{MF}} = 0.25$ keeps $r < t$ (the bootstrapped, finite-interval target), and the other 75% collapse to $r = t$, where (9) makes the loss *exactly* the FlowCast I-CFM regression. The diagonal supplies the clean, data-grounded anchor; the off-diagonal teaches the finite-interval average velocity that one-step sampling actually invokes. Channel weights and the `qpepre` log-PSD term are inherited unchanged from FlowCast.

5 Sampling

Starting from $z_0 \sim \mathcal{N}(0, I)$, integrate the learned average velocity over $[0, 1]$ in K equal jumps,

$$z_{(i+1)/K} \;=\; z_{i/K} + \tfrac{1}{K}\, u_\theta\big(z_{i/K},\, \tfrac{i}{K},\, \tfrac{i+1}{K};\, c_t\big), \qquad i = 0, \dots, K-1, \tag{12}$$

and reconstruct the field as $\widehat{X}_t = M_t + \sigma_{\text{data}}\, z_1$. The headline setting is $K = 1$ (one network evaluation, $\widehat{x}_1 = x_0 + u_\theta(x_0, 0, 1; c_t)$); $K = 2$ is the validation sweet spot. Unlike FlowCast's Euler sampler, raising K removes no truncation error — each jump is exact under a perfect u — it only shortens the intervals over which the *learned* u_θ must hold. Sampling uses the EMA weights.

Provenance and cross-references

The average-velocity objective and identity (7)–(11) are due to Geng et al. (*Mean Flows for One-step Generative Modeling*, arXiv:2505.13447, 2025); here they are mirrored into the StormCast residual convention so the diagonal collapses onto the FlowCast baseline. The same pixel-space MeanFlow objective has since been carried to autoregressive dynamical-system forecasting by Yang & Xue (*MeLISA: Towards Scalable One-step Generative Modeling for Autoregressive Dynamical System Forecasting*, arXiv:2605.05540, 2026), corroborating average-velocity sampling as a route to one-step rollouts for physical dynamics — the same motivation as this thesis.

In this thesis. The derivation and design choices above are Chapter 3, § *The MeanFlow Residual* (Eqs. for the average velocity, the identity, the loss, and the sampler) and § *Loss Design for the Precipitation Channel*; the background flow-matching theory MeanFlow generalises is Chapter 2, § *Flow Matching, FlowCast, and MeanFlow*. *In the code.* Loss `stormcast/utils/meanflow_loss.py` (`MeanFlowLoss`); training loop `stormcast/utils/trainer_meanflow.py`; preconditioner `stormcast/utils/meanf` sampler `meanflow_model_forward` in `stormcast/utils/nn.py`; configs `stormcast/config/{training,model}/me` long-form notes `stormcast/docs/meanflow.{tex,md}`.